

23. live 風補正ノード

solar_ws0129-main

2026-07-29

対象

項目	内容
ファイル	mpc_solarcar/wind_correction_node.py
種別	Python
区分	runtime node
役割	観測 headwind と現在距離から raw forecast CSV を補正し、corrected forecast CSV を書き出す。
起動文脈	live_wifi で planner 入力の風を上書きする前処理。

このファイルを読む理由

観測 headwind と現在距離から raw forecast CSV を補正し、corrected forecast CSV を書き出す。

- planner は /weather/headwind_corrected_ms を直接読むのではない。
- corrected CSV を mpc_node が再読込して効かせる。

呼び出し関係

どこから呼ばれるか

- launch/solar_race_live_wifi.launch.py

次に読むと理解が進むファイル

- mpc_solarcar/mpc_node.py

同一リポジトリ内の主要依存

- 同一リポジトリ内の明示 import は少ない。

ソース冒頭の把握

モジュール docstring または先頭意図の要約:

モジュール docstring は明示されていない。

代表コード断片

```
return int(pd.Timestamp(value).value)

class WindCorrectionNode(Node):
    def __init__(self) -> None:
        super().__init__("wind_correction_node")
        self.declare_parameter("forecast_csv", "")
        self.declare_parameter("corrected_forecast_csv", "outputs/runtime/
live_forecast_corrected.csv")
        self.declare_parameter("forecast_time_tz", "Australia/Darwin")
        self.declare_parameter("publish_period_sec", 30.0)
        self.declare_parameter("measurement_sigma_ms", 1.0)
        self.declare_parameter("correlation_distance_km", 300.0)
        self.declare_parameter("fallback_correlation_time_h", 3.0)
        self.declare_parameter("forecast_sigma0_ms", 1.5)
        self.declare_parameter("forecast_variance_growth_per_hour", 0.05)
        self.declare_parameter("planning_quantile", 0.5)
        self.declare_parameter("confidence_z", 1.96)
        self.declare_parameter("min_sigma_ms", 0.2)
        self.declare_parameter("preferred_source", "auto")
        self.declare_parameter("use_exp_distance_decay", True)

        self.forecast_csv = Path(str(self.get_parameter("forecast_csv").value or "")).
expanduser()
```

主要構造の読み取り

主要クラスは WindCorrectionNode。主要関数は main。ROS パラメータ宣言は 14 件。ROS I/O は publisher 2 件、subscription 2 件。

主要クラス・関数

行	種別	名前	補足
20	class	WindCorrectionNode	bases: Node
16	function	_timestamp_ns	args: value
21	function	__init__	args: self
64	function	_on_headwind	args: self, msg
67	function	_on_s	args: self, msg
70	function	_load_base	args: self
91	function	_interp_headwind_now	args: self, now_utc
105	function	_step	args: self
154	function	main	

ファイルを上から読んだときの定義順

行	内容
16	関数 _timestamp_ns を定義する。
20	クラス WindCorrectionNode を定義する。

import 群の詳細

行	import 文	このファイルで読む理由
1	<code>from__future_ _importannotations</code>	型ヒントの遅延評価を有効にし、自己参照型や forward reference を安全に扱うため。このファイル内での使用位置は少ないか、間接利用である。
3	<code>importmath</code>	<code>clip</code> 、 <code>sqrt</code> 、 <code>sin/cos</code> 、有限判定など数値ロジックに使うため。このファイル内での主な使用位置は L52, L53, L111, L121, L122, L130, L131, L135。
4	<code>importos</code>	パス、環境変数、プロセス外部状態を扱うため。このファイル内での主な使用位置は L72。
5	<code>fromdatetimeimportdatetime, timezone</code>	UTC 時刻や相対時間を扱うため。このファイル内での主な使用位置は L91, L109, L128。
6	<code>frompathlibimportPath</code>	ファイルやディレクトリを安全に扱うため。このファイル内での主な使用位置は L38, L39。
7	<code>fromstatisticsimportNormalDist</code>	<code>statistics</code> から <code>NormalDist</code> を読み込み、このファイルの処理を組み立てるため。このファイル内での主な使用位置は L50。
9	<code>importpandasaspd</code>	CSV や時刻列の読込・整形・再サンプリングに使うため。このファイル内での主な使用位置は L17, L77, L79, L96, L119, L128, L140, L147。
10	<code>importrcipy</code>	ROS 2 Python ノードとして起動・spin するため。このファイル内での主な使用位置は L155, L157, L159。
11	<code>fromrcipy.nodeimportNode</code>	ROS 2 ノード本体の基底クラスとして使うため。このファイル内での主な使用位置は L20。
13	<code>fromstd_msgs. msgimportFloat32,String</code>	<code>Float32</code> や <code>String</code> など軽量メッセージで <code>planner / vehicle</code> 値を publish するため。このファイル内での主な使用位置は L57, L58, L59, L60, L64, L67, L115, L151。

関数・クラスを上から順に解説

L16 関数 _timestamp_ns

- 定義: `_timestamp_ns(value)`
- docstring: 明示されていない。
- このブロックが直接呼ぶ主な関数・メソッド: `Timestamp`, `int`
- 戻り値の要点: `int(pd.Timestamp(value).value)`

1. `int(pd.Timestamp(value).value)` を返す。

L20 クラス WindCorrectionNode

- 定義: `WindCorrectionNode(bases=Node)`
- docstring: 明示されていない。

- このブロックが直接呼ぶ主な関数・メソッド: Float32, NormalDist, Path, String, Timedelta, Timestamp, __init__, _interp_headwind_now, _load_base, abs, any, append
 - 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
1. 関数 __init__ を定義する。
 2. 関数 _on_headwind を定義する。
 3. 関数 _on_s を定義する。
 4. 関数 _load_base を定義する。
 5. 関数 _interp_headwind_now を定義する。
 6. 関数 _step を定義する。

L154 関数 main

- 定義: main()
 - docstring: 明示されていない。
 - このブロックが直接呼ぶ主な関数・メソッド: WindCorrectionNode, destroy_node, init, shutdown, spin
 - 戻り値の要点: 戻り値が無いか、return 文が明示されていない。
1. rclpy.init(...) を実行する。
 2. node に WindCorrectionNode() の結果を代入する。
 3. rclpy.spin(...) を実行する。
 4. node.destroy_node(...) を実行する。
 5. rclpy.shutdown(...) を実行する。

設定値と入力パラメータ

行	名前	既定値・補足
23	forecast_csv	
24	corrected_forecast_csv	outputs/runtime/live_forecast_corrected.csv
25	forecast_time_tz	Australia/Darwin
26	publish_period_sec	30.0
27	measurement_sigma_ms	1.0
28	correlation_distance_km	300.0
29	fallback_correlation_time_h	3.0
30	forecast_sigma0_ms	1.5
31	forecast_variance_growth_per_hour	0.05
32	planning_quantile	0.5
33	confidence_z	1.96
34	min_sigma_ms	0.2
35	preferred_source	auto

ROS topic I/O

行	種別	topic	callback / 補足
57	publisher	/weather/headwind_corrected_ ms	publisher
58	publisher	/weather/wind_correction_ status	publisher
59	subscription	/weather/headwind_meas_ms	self._on_headwind
60	subscription	/vehicle/s_km	self._on_s

処理の流れ

1. 初期化時に設定値や入力パスを読み込む。
2. publisher / subscription / timer を準備する。
3. timer callback 周期で主処理を進める。

このファイルの位置づけ

この資料群では、`mpc_solarcar/wind_correction_node.py` を **runtime node** の中核として扱う。したがって、ここで扱う処理は単独で完結せず、前段の `profile / launch / telemetry` と後段の `planner / logger / report` へ繋がる。